

Homework 03

Rahaf Alabdullah (rahafaa2)

2026-09-13

Question 1: explain-to-me skill

The skill helps explain a homework question without completing it. It restates the goal, identifies the required work, defines unfamiliar terms, points to the relevant Week 3 lecture, and asks a check-for-understanding question. It does not provide a complete solution unless explicitly requested.

Question 2: Nearly collinear predictors

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(43231)
n= 100
p=4
reptions= 200
sigma= 1.0
L_i = rng.normal(loc=0.0, scale=1.0, size=n)
U_i1 = rng.normal(loc=0.0, scale=1.0, size=n)
U_i2 = rng.normal(loc=0.0, scale=1.0, size=n)
X3 = rng.normal(loc=0.0, scale=1.0, size=n)
X4 = rng.normal(loc=0.0, scale=1.0, size=n)
X1= L_i + 0.04 * U_i1
X2= L_i + 0.04 * U_i2
X_raw= np.column_stack((X1, X2, X3, X4))
x_bar = np.mean(X_raw, axis=0)
X = X_raw - x_bar
X = X / np.sqrt(np.mean(X ** 2, axis=0))
beta= np.array([1.5, 1.5, 1, 0])
mu= X @ beta
lambda_values=np.array([0.0, 0.02, 0.2])

singular_values = np.linalg.svd(X, compute_uv=False)
largest_sv = np.max(singular_values)
smallest_sv = np.min(singular_values)
condition_number = (largest_sv / smallest_sv) ** 2
print(f"Largest singular value: {largest_sv:.3f}")
print(f"Smallest singular value: {smallest_sv:.3f}")
print(f"Condition number: {condition_number:.3f}")
cor_x1_x2 = np.corrcoef(X[:, 0], X[:, 1])[0, 1]
print(f"Cor(x1, x2): {cor_x1_x2:.3f}")
```

```

intercept_draws_py = np.empty(repetitions)
gram = X.T @ X
train_mse_draws_py = np.empty((repetitions, len(lambda_values)))
test_mse_draws_py = np.empty((repetitions, len(lambda_values)))
beta_hat = np.empty((repetitions, p, len(lambda_values)))
for k in range(repetitions):
    y_train= mu + rng.normal(loc=0.0, scale=sigma, size=n)
    y_test_star= mu + rng.normal(loc=0.0, scale=sigma, size=n)
    y_bar= y_train.mean()
    y_centered= y_train - y_bar
    intercept_draws_py[k] = y_bar

    for i, lam in enumerate(lambda_values):
        beta_hat[k, :, i] = np.linalg.solve(
            gram + n * lam * np.eye(p),
            X.T @ y_centered,
        )
        y_hat = y_bar + X @ beta_hat[k, :, i]
        train_mse_draws_py[k, i] = np.mean((y_train - y_hat) ** 2)
        test_mse_draws_py[k, i] = np.mean((y_test_star - y_hat) ** 2)

beta_sum = beta_hat[:, 0, :] + beta_hat[:, 1, :]
beta_difference = beta_hat[:, 0, :] - beta_hat[:, 1, :]
summary_rows = []
for i, lam in enumerate(lambda_values):
    summary_rows.append({
        "lambda": lam,
        "beta1": beta_hat[:, 0, i].mean(),
        "beta2": beta_hat[:, 1, i].mean(),
        "beta1 + beta2": beta_sum[:, i].mean(),
        "beta1 - beta2": beta_difference[:, i].mean(),
        "train_mse": np.mean(train_mse_draws_py[:, i]),
        "test_mse": np.mean(test_mse_draws_py[:, i]),
    })

df_summary = pd.DataFrame(summary_rows).round(3)
print(df_summary.to_string(index=False))

```

Largest singular value: 14.173

Smallest singular value: 0.406

Condition number: 1218.061

Cor(x1, x2): 0.998

lambda	beta1	beta2	beta1 + beta2	beta1 - beta2	train_mse	test_mse
0.00	1.290	1.701	2.992	-0.411	0.949	1.038
0.02	1.465	1.497	2.961	-0.032	0.960	1.029
0.20	1.355	1.359	2.715	-0.004	1.064	1.125

Question 3: What does ridge regression shrink?

The objective is

$$L_\lambda(\beta) = \frac{1}{2n} \|\tilde{y} - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2.$$

Differentiating gives

$$\nabla L_\lambda(\beta) = \frac{1}{n} X^T (X\beta - \tilde{y}) + \lambda\beta.$$

Setting this equal to zero and multiplying by n gives

$$(X^T X + n\lambda I_p) \hat{\beta}_\lambda = X^T \tilde{y}.$$

For any nonzero v ,

$$v^T \left(\frac{1}{n} X^T X + \lambda I_p \right) v = \frac{1}{n} \|Xv\|_2^2 + \lambda \|v\|_2^2 > 0$$

when $\lambda > 0$. Thus the curvature matrix is positive definite and the minimizer is unique even when X is rank deficient or $p > n$.

With $X = UDV^T$, the fitted values are

$$X\hat{\beta}_\lambda = \sum_{j=1}^r \frac{d_j^2}{d_j^2 + n\lambda} u_j u_j^T \tilde{y}.$$

For $n = 100$, $\lambda = 0.01$:

$$\rho_1 = \frac{100}{101} = 0.9901, \quad \rho_2 = \frac{4}{5} = 0.8, \quad \rho_3 = \frac{0.04}{1.04} = 0.0385.$$

Therefore,

$$\text{df}_{\text{eff}} = 1 + \rho_1 + \rho_2 + \rho_3 \approx 2.8286.$$

The $d_3 = 0.2$ direction receives the strongest shrinkage because it has the smallest singular value. This reduces prediction variance because noise is amplified in a weakly identified direction. It can create substantial bias when the true signal has an important component in that same direction.

Question 4: Ridge optimization

```
import numpy as np
import pandas as pd
eigen_array= np.array([9, 0.01])
lambda_value= 0.09
eig_lam= eigen_array + lambda_value
min_eig_lam= np.min(eig_lam)
max_eig_lam= np.max(eig_lam)
condition_number= max_eig_lam / min_eig_lam
print(f"Condition number: {condition_number:.3f}")

def contraction_factor(eta, lambda_value, a):
```

```

    factor = 1-eta * (lambda_value + a)
    return factor
M=9.09
eta1= 1/M
eta2= 2.1/M
factor1= contraction_factor(eta1, lambda_value, eigen_array[0])
factor2= contraction_factor(eta1, lambda_value, eigen_array[1])
print(f"Contraction factor for eta = 1/M and largest eigenvalue: {factor1:.3f}")
print(f"Contraction factor for eta = 1/M and smallest eigenvalue: {factor2:.3f}")
factor3= contraction_factor(eta2, lambda_value, eigen_array[0])
factor4= contraction_factor(eta2, lambda_value, eigen_array[1])
print(f"Contraction factor for eta = 2.1/M and largest eigenvalue: {factor3:.3f}")
print(f"Contraction factor for eta = 2.1/M and smallest eigenvalue: {factor4:.3f}")

```

Condition number: 90.900

Contraction factor for eta = 1/M and largest eigenvalue: 0.000

Contraction factor for eta = 1/M and smallest eigenvalue: 0.989

Contraction factor for eta = 2.1/M and largest eigenvalue: -1.100

Contraction factor for eta = 2.1/M and smallest eigenvalue: 0.977

Question 5: Selecting the ridge penalty

The code below performs fold-specific scaling, cross-validation, GCV, and final evaluation. The two plots are generated from the same calculations.

```

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

here = Path.cwd()
data = pd.read_csv(here / "data" / "realestate.csv")
splits = pd.read_csv(here / "data" / "realestate-split-folds.csv")
data = data.merge(splits, on="row_id", validate="one_to_one").sort_values("row_id")
features = ["date", "age", "distance", "stores", "latitude", "longitude"]
lambdas = np.r_[0.0, 10 ** (-4 + 0.05 * np.arange(121))]
train = data[data.split == "train"].copy()
test = data[data.split == "test"].copy()

fold_errors = np.empty((10, len(lambdas)))
for fold_index, fold in enumerate(range(1, 11)):
    fit = train.cv_fold != fold
    valid = ~fit
    X_fit = train.loc[fit, features].to_numpy()
    X_valid = train.loc[valid, features].to_numpy()
    y_fit = train.loc[fit, "price"].to_numpy()
    y_valid = train.loc[valid, "price"].to_numpy()
    center = X_fit.mean(0)
    scale = np.sqrt(np.mean((X_fit - center) ** 2, axis=0))
    X_fit = (X_fit - center) / scale
    X_valid = (X_valid - center) / scale
    y_bar = y_fit.mean()

```

```

gram = X_fit.T @ X_fit
for j, lam in enumerate(lambdas):
    coef = np.linalg.solve(gram + len(y_fit) * lam * np.eye(6),
                           X_fit.T @ (y_fit - y_bar))
    fold_errors[fold_index, j] = np.mean(
        (y_valid - (y_bar + X_valid @ coef)) ** 2)
mean_cv = fold_errors.mean(0)
se_cv = fold_errors.std(0, ddof=1) / np.sqrt(10)
min_index = np.argmin(mean_cv)
threshold = mean_cv[min_index] + se_cv[min_index]
eligible = np.flatnonzero(mean_cv <= threshold + 1e-12)
index_1se = eligible[np.argmax(lambdas[eligible])]
lambda_min = lambdas[min_index]
lambda_1se = lambdas[index_1se]

plt.figure(figsize=(5.8, 3.1))
plt.semilogx(lambdas[1:], mean_cv[1:], color="black")
plt.axhline(threshold, color="gray", ls="--", label="1-SE threshold")
plt.axvline(max(lambda_min, lambdas[1]), color="steelblue", ls=":", label="lambda min")
plt.axvline(max(lambda_1se, lambdas[1]), color="crimson", ls="-. ", label="lambda 1se")
plt.xlabel("lambda"); plt.ylabel("mean validation MSE")
plt.legend(fontsize=7); plt.tight_layout(); plt.show()

X_train = train[features].to_numpy(); y_train = train.price.to_numpy()
X_test = test[features].to_numpy(); y_test = test.price.to_numpy()
center = X_train.mean(0); scale = np.sqrt(np.mean((X_train-center)**2, 0))
X_train = (X_train-center)/scale; X_test = (X_test-center)/scale
n = len(y_train); y_bar = y_train.mean(); gram = X_train.T @ X_train
singular_squared = np.linalg.svd(X_train, compute_uv=False) ** 2
df_eff = 1 + np.array([np.sum(singular_squared /
                             (singular_squared + n * lam)) for lam in lambdas])
train_mse = np.empty(len(lambdas))
for j, lam in enumerate(lambdas):
    coef = np.linalg.solve(gram + n * lam * np.eye(6),
                           X_train.T @ (y_train-y_bar))
    train_mse[j] = np.mean((y_train-y_bar-X_train@coef)**2)
gcv = train_mse / (1 - df_eff/n) ** 2
lambda_gcv = lambdas[np.argmin(gcv)]

plt.figure(figsize=(5.8, 3.1))
plt.semilogx(lambdas[1:], gcv[1:], color="black")
plt.axvline(max(lambda_gcv, lambdas[1]), color="green", ls="--", label="lambda GCV")
plt.xlabel("lambda"); plt.ylabel("GCV")
plt.legend(fontsize=7); plt.tight_layout(); plt.show()

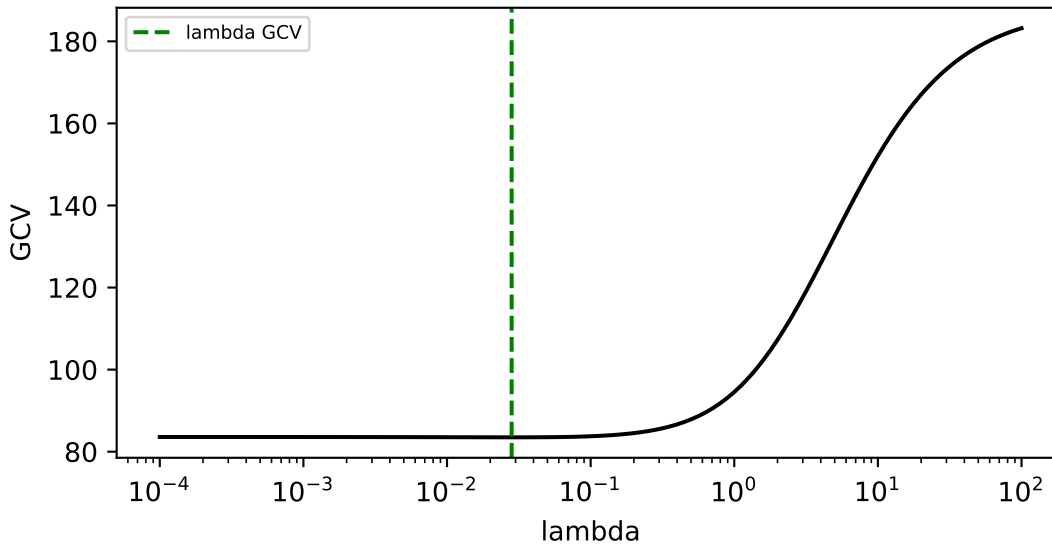
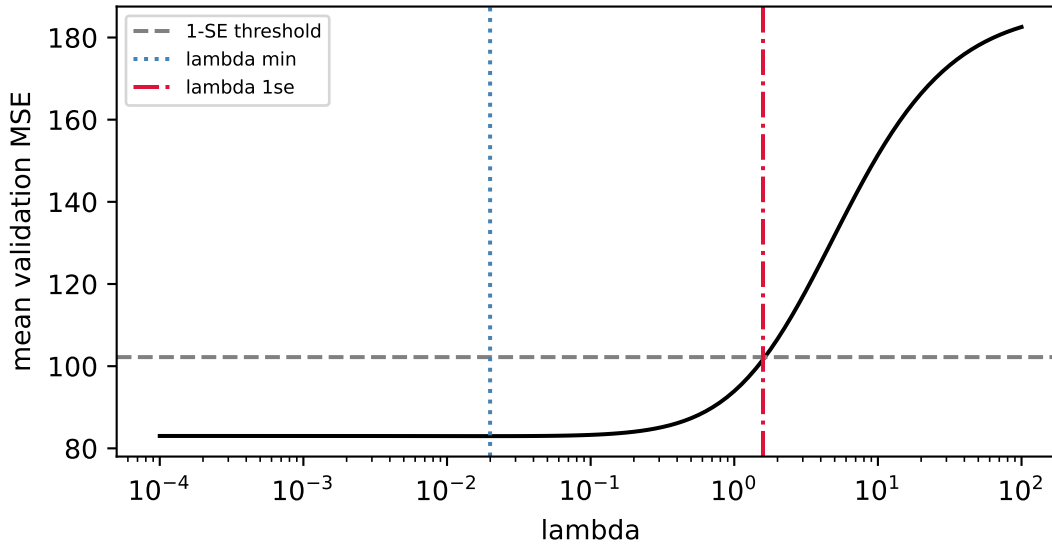
choices = [("OLS", 0.0), ("CV min", lambda_min),
           ("1-SE", lambda_1se), ("GCV", lambda_gcv)]
results = []
for name, lam in choices:
    coef = np.linalg.solve(gram + n * lam * np.eye(6),
                           X_train.T @ (y_train-y_bar))
    pred_train = y_bar + X_train @ coef

```

```

pred_test = y_bar + X_test @ coef
results.append({"model": name, "lambda": lam,
               "df_eff": 1 + np.sum(singular_squared /
                                   (singular_squared+n*lam)),
               "train_mse": np.mean((y_train-pred_train)**2),
               "test_mse": np.mean((y_test-pred_test)**2)})
q5_results = pd.DataFrame(results).round(2)
q5_results

```



	model	lambda	df_eff	train_mse	test_mse
0	OLS	0.00	7.00	80.09	66.32
1	CV min	0.02	6.77	80.14	66.00
2	1-SE	1.58	3.03	100.37	81.20
3	GCV	0.03	6.69	80.17	65.90

Cross-validation selects $\lambda_{\min}$ by minimizing estimated validation error. The one-SE choice deliberately accepts a nearby error in exchange for more shrinkage. GCV uses a full-training smoother and effective degrees of freedom,

so its selected penalty may differ from fold-based CV. The final test set is used only after all choices are fixed; therefore its smallest observed MSE cannot be used to choose a model without contaminating the evaluation.

explain-to-me skill

```
---
name: explain-to-me
description: "Use when Rahaf explicitly asks to use explain-to-me to understand a STAT 432 homework"
---

# Explain to Me

When I explicitly ask you to use this skill, help me understand the homework question without completing it.

- Restate the question in clear, simple language.
- Identify the main goal and list the work the question requires.
- Define unfamiliar statistical or mathematical terms briefly.
- Point to the most relevant Week 3 lecture section:
  - Ridge Regression: Stability Through Shrinkage
  - From a Penalized Objective to a Fitted Ridge Model

Do not calculate the final answer, write the complete code, or solve any requested derivation unless explicitly asked.

## Personal preferences

Replace this section with your own preferences for explanations.

- Preferred level of detail: [give me general idea. If I need more detail, I will ask for it.]
- Preferred language or examples: [Examples preferred to be in Python]
- Topics that usually need extra explanation: [It is ok to provide additional context or background]
```